CSC3050 Project 4: Cache Simulation

Danyang Chen
123090018@link.cuhk.edu.cn

Yuhao Liu
224040365@link.cuhk.edu.cn

April 8, 2025

1 Introduction

This project is designed to be completed **in groups of 2 students**.

It focuses on simulating the cache to provide hands-on experience with cache systems and their role in improving system performance. The cache simulator serves as a tool for analyzing cache behavior within the memory hierarchy, enabling both single-level and multi-level cache simulation to evaluate performance under different configurations.

The main features of the simulator include:

- Supports for both single-level and multi-level cache structures.
- Configurable cache parameters such as size, block size (or cache line size), and associativity.
- Performance evaluation metrics, including miss rate and total cycles.
- Trace-based memory access simulation to analyze cache performance.
- Implements **Write Back**, **Write Allocate**, and **Least Recently Used (LRU)** replacement policy to manage cache updates and evictions efficiently.

To facilitate cache simulation, memory access traces (Trace) are used to capture the memory access patterns of a program. Each trace entry follows the format:

- Each line represents one memory access.
- Format: `<op> <address>`, where `op` indicates the operation (e.g., `r` for read, `w` for write), and `address` specifies the memory location accessed.
- The memory trace is processed sequentially to simulate cache operations.

Note: This project consists of 4 independent parts. You are allowed to work on any part individually. If you are unable to complete one part, you can still attempt and complete the others.

2 How to run the starter code

The official starter code is hosted at:
https://github.com/DanielDanyang/CSC3050-2025-Spring-Project-4

If you plan to **create your own repository** based on this code, please make sure to set your repository to **private**. This is to avoid academic integrity issues and to protect your code from being accessed publicly.

We will use the docker of Project 3 as our environment. The file tree in this folder is as follows:

- include
  - `Cache.h` — Core cache system class definitions
  - `Debug.h` — Debugging utility functions
  - `elfio` — (Can be ignored)
  - `MemoryManager.h` — Memory management
  - `MultiLevelCacheConfig.h` — Multi-level cache configuration parameters
- src
  - `Cache.cpp` — Implementation of cache system functionality
  - `MainMulCache.cpp` — Multi-level cache simulator entry point
  - `MainSinCache.cpp` — Single-level cache simulator entry point
  - `MemoryManager.cpp` — Implementation of memory management system
- trace
  - `Part1` — trace files used in Part 1
  - `Part2` — trace files used in Part 2
  - `Part3` — trace files used in Part 3
  - `Part4` — trace files you generate in Part 4
- `CMakeLists.txt` — Project build configuration file
- `PINTool.tar.gz` — Will be introduced in Part 4
- `README.md`
- `report.md` — report template

The file used to declare the rule of compiling the C++ program into an executable is CMakeLists.txt. You can open it to check the specific structure. For more details about using CMake, you can refer to Google. Then we create a new folder *build* and enter it:

```
mkdir build

cd build
```

Next, we will compile the C++ program into an executable:

```
cmake ..

make
```

In the CMakeLists.txt, we actually define the rule for creating two executables named **CacheSingle** and **CacheMulti**, which are the single-level simulator and multi-level simulator, respectively.

Finally, we will run the executable. Here, we use the single-level simulator as an example. The running command will be

```
./CacheSingle ../trace/Part2/test.trace
```

where the only parameter is the address of the test trace and you will see something at end like this:

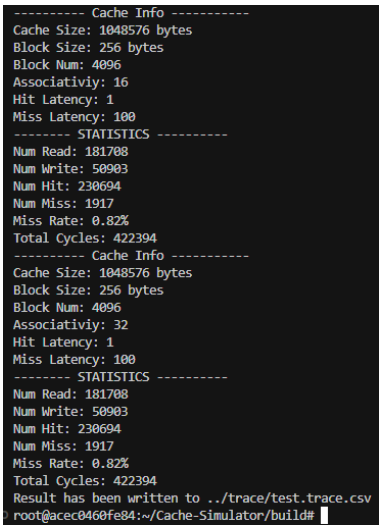


Figure 1: Running the single-level cache simulator

3 Part 1: Split Cache Simulation

The original starter code uses a **Unified Cache** as the Level 1 cache, which refers to a cache design where both instructions and data are stored in the same cache. In the current architecture, at level 1 cache, it is common to use **Split Cache** to store instructions and data separately into two caches(i.e. ICache and DCache) with the same size. You should implement the Split cache design in this part.

3.1 Split vs. Unified Cache Trade-offs

There are some important trade-offs between Split and Unified cache architectures:

1. **Bandwidth Benefits of Split Cache**. Split cache architecture allows simultaneous access to instructions and data. This parallel access capability significantly reduces total cycle count compared to Unified cache, especially for some instruction dense tasks. Also, it can alleviates cache bandwidth pressure by eliminating instruction-data access conflicts. Therefore, it is particularly beneficial for superscalar processors that need to fetch multiple instructions while executing memory operations.

2. **Capacity Constraints and Miss Rates.** Split cache divides the total cache capacity between instruction and data caches. So the limited capacity per cache could lead to higher miss rates compared to Unified cache. In contrast, Unified cache offers more flexible space allocation between instructions and data.

3.2 Simplifying Assumptions

To maintain educational clarity and programming feasibility, we would give two main simplifying assumptions:

1. **Parallel Access in Split Cache**. It's a rough simulation and not really Parallel Access but we just treat them as if they were accessed in parallel. In order to make them "run in parallel", the total cycle count of Split Cache is determined by **taking the maximum between ICache and DCache cycles**. In contrast, the Unified Cache simply sums up all access cycles.

2. **Fixed Miss Penalty**. The starter code uses a constant cycle penalty for cache misses. This simplification doesn't account for various structural hazards or conflicts that might occur in real systems. Real cache systems would have variable miss penalties depending on multiple factors (memory hierarchy levels, bus conflicts, etc.) but you don't need to consider them within this assignment.

3.3 Some Implementation details

1. At the most cases, we cannot modify instructions, so for the ICache, we just need to implement the read policies(i.e. Read Allocate).

2. We get instructions via ICache only, and access data from DCache.

3. For ICache, since it only handles instruction fetches (read-only operations), there is no need to consider write-back issues. However, in the Unified Cache scenario, when using a write-back policy, data blocks require additional cycles for memory updates during replacement. So this write-back penalty adds to the total cycle count.

3.4 Implementation Requirements and Evaluation

Based on the starter code and above introduction, you are required to implement a *ICache* class for simulating a single-level split cache. You should write your code in `src/MainSinCache.cpp`.

Cache Type	Cache Size	Block Size	Associativity	Write Policy
Split Cache	8KB (single one)	64 B	1 way	-
Unified Cache	16KB	64 B	1 way	write back and write allocate

Table 1: Split and Unified Cache parameters

- **Configuration:** The configuration parameters for Split Cache and Unified Cache are shown in Table 1. You **cannot** modify them. ICache and DCache usually have the same capacity, and the total capacity of Split Cache should be the same as the one of Unified Cache. The simulated cache should be able to perform correct read policies and cycle count.

  In order to reduce the number of code files, you can run simulations for Split cache and Unified cache within the same code file which would call all necessary classes.

- **Input and Output:** The format of test trace for this part would be like: `<op> <address> <type>`, where `op` indicates the operation (e.g., `r` for read, `w` for write), and `address` specifies the memory location accessed, while `<type>` shows whether it is an instruction(I) or an element(D). You may need to modify the corresponding code to fit this input format.

  We will provided two short test trace files: `trace/Part1/I.trace`, which includes a lot of Instruction access, and `trace/Part1/D.trace` which includes a lot of Data access.

  After feeding trace to your simulator, the total number of miss and total cycles need to be saved in a CSV file(The starter code has provided this feature).

- **Performance Evaluation:** For the Split cache, the total miss count should be the sum of ICache and DCache miss, while the total cycle count is the maximum between ICache and DCache cycles. In order to verify the correctness of Split cache, we have provided two short test trace files and expected statistic. Your performance should be the same as the figures in the Table 2.

Trace	Split or Unified	# Miss	Total Cycles
I.trace	Split	512	41088
I.trace	Unified	384	54912
D.trace	Split	634	67584
D.trace	Unified	507	68141

Table 2: Performace for Split Cache and Unified Cache

4 Part 2: Multi-Level Cache Simulation

We have offered you an initial multi-level cache based on the single-level Unified cache simulator. The configuration is in `include/MultiLevelCacheConfig.h`. You can run it by:

Level	Num Reads	Num Writes	Num Hits	Num Misses	Miss Rate	Total Cycles
L1	181,708	50,903	177,911	54,700	23.5%	717,519
L2	47,667	7,033	25,574	29,126	53.1%	888,292
L3	26,984	2,142	21,596	7,530	25.9%	1,184,920

Table 3: Multi-Level Cache Simulation Results (Cycles may vary within 10% tolerance)

```
./CacheMulti ../trace/Part2/test.trace
```

Important: You are **not allowed to modify** `include/MultiLevelCacheConfig.h` in this part.

There is an issue with the hit/miss counting method in the multi-level cache system. For any cache level, we should only count the hits/misses specific to that level. For example:

- When there is an L2 miss, only L2 miss count should be incremented by 1 and L1 count should not be affected. If an L3 hit occurs subsequently, only L3 hit count should be incremented by 1 and L2 hit count should not be affected.
- L1 Cache miss = L2 hit + L2 miss (The total number of L1 misses should equal the sum of L2 hits and L2 misses).

You need to fix the current statistics counting method to ensure each cache level only counts its own hits and misses.

The input for this part is `trace/Part2/test.trace`, and you should save all three levels' statistic into a CSV file. The correct results for this test are shown in Table 3. Note that the total cycles (**Total Cycles**) may vary within a 10% tolerance.

5 Part 3: Trace-Driven Cache Optimization

We will provide you with three techniques and three trace files (*test1.trace*, *test2.trace*, and *test3.trace*). This part is a **blind test**, which means you **won't** know which technique works best for each trace file in advance. Your task is to analyze memory access patterns and implement optimization techniques to improve performance of those trace. While we use Multi-Level Cache as the base system, we **ONLY** focus on optimizing the L1 Cache and care about L1 cache performance.

5.1 Application-Specific Techniques

Here are three techniques to learn:

1. **Pre-fetching**

   Implement the pre-fetching technique to fetch data in advance based on detected memory access patterns. You will implement a pre-fetching algorithm capable of detecting

fixed-stride memory access patterns; the pseudo-code of the algorithm is summarized in Algorithm 1.

Algorithm 1 Stride-Based Pre-fetching Algorithm

```
initialize: stride = 0, is_prefetch = false.
for Each Memory Access do
    Calculate the memory access stride (the distance between the current memory access address and the address of the previous memory access with the same operation).
    if is_prefetch = false and there are more than three times with the same stride then
        is_prefetch = true
        prefetch_address = current_address + stride
        Prefetching(prefetch_address)
    end if
    if is_prefetch = true and more than three times the different strides are detected then
        is_prefetch = false.
        Stop prefecting.
    end if
end for
```

2. **FIFO**

   You may need to use FIFO as the replacement policy although we use LRU in the provided code. Notice the FIFO **does not** work for Direct-Mapped because there is only one position for each set, and if different addresses have the same Set Index, we don't have any other place to choose. So in order to make FIFO effective, you also need to modify the configuration of L1 cache, which should be fully-associative.

3. **Victim Cache**

   A victim cache is a small, fully-associative auxiliary cache that stores blocks evicted from the main cache. When a miss occurs in the main cache(L1 cache), the victim cache is checked. If a hit is found in the victim cache, the blocks in the main cache and victim cache are swapped. If both of them miss, we have to get data from the next layer(lower cache or memory), and store it into main cache, while the evicted block from main cache should be saved into Victim Cache.

   For simplicity, the statistics for victim cache are calculated as follows: you can directly subtract the victim cache hit number from the main cache miss number to get the final main cache miss number(**the evaluation would use this way**). However, please note that the total number of memory accesses for the main cache should remain unchanged.

5.2 Implementation Requirements and Evaluation

Table 4 shows how L1 cache fits well with different techniques.

Techniques	L1 Cache configuraion
Prefetching	Direct Mapped
FIFO	Fully Associative
Victim Cache	Direct Mapped

Table 4: Techniques and Cache Configurations

- **Configuration**
  - **Basic cache configuration:** The basic parameters for L1 are shown in Table 5, you **cannot** modify them.

Cache Size	Block Size	Write Policy
16 KB	64 B	write back and write allocate

Table 5: Basic L1 Cache Configuration

  - **Technique parameter configuration:** If you want to implement Victim Cache, please use the configuration shown in Table 6 (DO NOT modify it). Besides, you can directly subtract the victim cache hit number from the main cache miss number to get the final L1 cache miss number.

Parameter	Values
Cache Size	8 KB
Block Size	64 B
Associativity	Fully Associative

Table 6: Parameters for Victim Cache

- **Input and Output:** Input would be one of the trace files, and you should save the L1 Cache miss rate **ONLY** into a CSV file, which also includes the name of trace file you are testing.
- **Evaluation:** In order to verify the correctness of your choice, we have provided expected statistic for each trace file. You need to compare your results with the ones in Table 7.
- **Requirements:**
  - Your implementation should support enabling these optimizations through command-line parameters. For example:

```
./CacheMulti -p ../trace/Part3/test1.trace # prefetching
./CacheMulti -f ../trace/Part3/test1.trace # fifo
./CacheMulti -v ../trace/Part3/test1.trace # victim
```

Trace file	Expected miss rate
test1.trace	0.371%
test2.trace	5.803%
test3.trace	0.372%

Table 7: Expected Miss Rates for Different Trace Files

- You must implement at least two of the three optimization techniques and get the similar performance(less than 10% difference) to receive full credit
- Implementing all three techniques will earn you a 5-point bonus

Hint: For some trace, you may find both Victim Cache and FIFO can save performance, but FIFO may not work for the trace tailored to Victim Cache. So you'd better test more by yourself.

6 Part 4: Matrix Multiplication Performance Analysis Using PINTool

Before starting this part, you need to make the following modifications:

- In `include/MultiLevelCacheConfig.h`, line 10, change:

```
policy.cacheSize = 16 * 1024;  // Change this to:
policy.cacheSize = 4  * 1024;
```

- In `include/MultiLevelCacheConfig.h`, line 13, change:

```
policy.associativity = 1;  // Change this to:
policy.associativity = policy.blockNum;
```

6.1 Generate Memory Access Trace Using PINTool

`PIN` is a runtime binary instrumentation tool that enables efficient, real-time analysis of memory accesses. It inserts code into a running application to track memory behavior and generate cache traces with minimal program modification.

Important Notes:

- The provided PINTool only traces `double` data type operations. Since `int` operations are less expensive in modern processors, they are not included in the trace.
- Do not delete the `register` keyword before `double` declarations (e.g., `register double cij`) in the matmul function. Removing this keyword will affect the correctness of the trace.

First, let's generate a memory access trace for the matrix multiplication implementation:

1. Extract the provided PINTool package:

```
tar -xzvf PINTool.tar.gz
```

2. Install required packages for 32-bit compilation:

```
dpkg --add-architecture i386
apt-get update
apt-get install g++-multilib libc6-dev-i386
```

3. Navigate to the examples directory where both `funcMemTrace.cpp` and `gemm.cpp` are located:

```
cd PINTool/source/tools/ManualExamples/
```

4. Compile the PINTool and test program:

```
mkdir obj-ia32
make obj-ia32/funcMemTrace.so TARGET=ia32
g++ -m32 -o gemm gemm.cpp
```

5. **Important:** To generate different traces for each matmul function, you need to modify the function call in the main function of `gemm.cpp` (Now you don't need to do anything):

```
// To generate trace for matmul0:
matmul0(C, A, B, n);
// For matmul1, comment out the above line and use:
// matmul1(C, A, B, n);
// matmul2(C, A, B, n);
// matmul3(C, A, B, n);
// matmul4(C, A, B, n);
```

6. Run the program with PIN to generate the trace:

```
../../../pin -t obj-ia32/funcMemTrace.so -- ./gemm
```

   This will generate `funcTrace.out` containing the memory access trace of the matrix multiplication function.

7. Copy the generated trace to your cache simulator's test directory:

```
cp funcTrace.out ../../../../trace/Part4/matmul0.trace
```

8. Open a new terminal and run the trace you generated:

```
cd build
./CacheMulti ../trace/Part4/matmul0.trace
```

6.2 Matrix Multiplication Implementation Analysis

Your tasks are:

1. **matmul0 and matmul1**
   - Generate memory access traces for both implementations
   - Save the trace as `matmul0.trace` or `matmul1.trace`
   - Run the traces through your cache simulator
   - Record their performance metrics in the report
2. **Implement Additional Versions**
   - `matmul2`: Implementation with `kij` loop order
   - `matmul3`: Implementation with `jki` loop order

   For each new implementation:
   - Add the new function to `gemm.cpp`
   - Generate its memory access trace using PINTool
   - Save the trace as `matmul2.trace` or `matmul3.trace`
   - Compare its performance with previous implementations
   - Analyze the performance differences based on your understanding of memory hierarchy
3. **Bonus Task: Implement matmul4**

   You may implement a new matrix multiplication function `matmul4`.
   - The goal is to optimize the L1 cache cycle count to be below **1 million cycles**.
   - Save the trace as `matmul4.trace` and record the performance.

   Hint: Use `ijk`-ordered blocked matrix multiplication to exploit spatial and temporal locality more efficiently.

6.3 Performance Analysis Requirements

For analyzing the performance of different matrix multiplication implementations:

- Use the unoptimized version of CacheMulti (without any optimization flags)
- Focus only on L1 cache performance metrics in your analysis
- Follow the report template structure provided in `report.md`

7 Submission

For this project, you need to submit the following files:

- src/*: include all source code files
- include/*: include all header files
- CMakelists.txt: the cmake file for your project
- trace/*: provided trace and the matmul trace you generate
- report.pdf: a detailed report following the template

Deadline: 2025/04/30, 23:59 PM. Please compress all files into a single zip file and submit it to the BlackBoard. The file name should be your student ID, like 122090000&123090000.zip. Only one student of a group need to submit the project.

8 Grading Details

The overall score will be calculated as follows:

- Part 1: 25 points
- Part 2: 25 points
- Part 3: 40 points + 5 bonus points
- Part 4: 10 points + 3 bonus points

Late Policy: For each day after the deadline, 10 points will be deducted from your final score up to 30 points, after which you will get 0 point.